Acre Audit

Côme du Crest

2025-08-19

Table of contents

- Table of contents
- Acre Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [High][Fixed] stBTC migration miss-calculates deposited assets
  - [High][Fixed] WithdrawalQueue requests could be stuck due to tBTCvault upgrade
  - [Med][Fixed] Incorrect fee on acreBTC.redeem and acreBTC.redeemAndBridge
  - [Low][Ack] Migration from stBTC to acreBTC may incur entry fee twice
  - [Low][Ack] Limitation on shares during migration is broken in fungible stBTC
  - [Info] acreBTC is fungible
  - [Info] Non-fungible vaults can be made fungible with a wrapper contract
  - [Info] Updating the fees should give enough notice for users to react
  - Optimisations and miscellaneous

Acre Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest for Acre.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes updated contracts within `acre-btc/acre` as of commit `0x27c5f5f`. The contracts have been audited at commit `0x965f38b` so the updates since the last commit have been reviewed (`see diff`).

The scope also includes the adapter contract for Acre written by Midas Protocol `AcreAdapter.sol` as of commit `0x15525a8`.

The report has been received by developers and fixes have been implemented. The updated code has been reviewed in `PR 968` as of commit `0xefdae0a`.

Context

Acre allows users to generate yield out of bitcoin via tBTC tokens. Tokens can be deposited into `acreBTC` that will use a dispatcher to send out the tBTC tokens to a Midas vault. Users can later withdraw their tBTC with a surplus.

The changes can be summarized by these contracts:

- `AcreMultiAssetVault.sol` a new wrapper contract on top of `MezoPortal` that allows to deposit different assets
- `BitcoinDepositor.sol` has been updated to include a bridge fee reimbursement scheme
- `FeesReimbursementPool.sol` a new contract to hold the assets used to reimburse the bridge fees by `BitcoinDepositor`
- `BitcoinRedeemer.sol` has been updated to add a check on `tbtcRedemptionData` redeemer
- `MezoAllocator.sol` has been updated with partial withdrawals from `MezoPortal`
- `PausableOwnable.sol` includes a new error when attempting to update to the same pause admin
- `acreBTC.sol` is an updated version of `stBTC` without the debt mechanism and with a withdrawal queue necessary for midas integration
- `ERC4626Fees.sol` has a new `maxWithdraw()` function

- `ERC4626NonFungibleWithdrawals.sol` a new contract to disable fungibility of the inheriting vault tokens
- `MidasAllocator.sol` a new contract to deposit the tBTC held by acreBTC into the midas vault
- `WithdrawalQueue.sol` a new contract to handle withdrawal requests from the midas vault
- `stBTC.sol` has been updated to have optionally non-fungible vault tokens, to have a debt mechanism, and migration functions for `acreBTC`
- `Maintainable.sol` a new contract providing the maintainer role

The contract changes following the reception of this audit are as follow:

- `BitcoinDepositor.sol` the fee reimbursement scheme has been removed and moved to `BitcoinDepositorV2.sol`
- `BitcoinDepositorV2.sol` has been moved from the `test/upgrades/` directory to `contracts/` and the fee reimbursement scheme has been added. It does not have the same storage layout as the previous version but the goal is not to upgrade from V1 to V2.
- `BitcoinRedeemerV2.sol` a new version of `BitcoinRedeemer.sol`. It can be called via `acreBTC.approveAndCall()` to redeem acreBTC share for tBTC and request bridging to bitcoin via `acreBTC.requestRedeemAndBridge()`.
- `acreBTC.sol` the `withdraw()` flow has been removed, now there is only `requestRedeem()` and `requestRedeemAndBridge()` that always go through the withdraw queue instead of withdrawing available assets if possible.
- `MidasAllocator.sol` variables and functions renaming
- `WithdrawalQueue.sol` rework of the fee mechanism to take the exact fee supplied by caller (`acreBTC`), add minimum amount in withdrawal request to work with the bitcoin bridge and add an admin function to update the tBTCVault.
- `stBTC.sol` added documentation to warn about potential issues during migration. Correctly calculate the amount of assets to deposit during migration. Added an event emitted during migration.

Status

The report has been sent to the protocol team.

The report has been reviewed and no issues remain. Some info level comments remain unanswered but they can safely be ignored.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Acre (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[High][Fixed] stBTC migration miss-calculates deposited assets

Summary

The function `stBTC.migrateDeposit()` calculates the amount of assets to be migrated by converting to assets the total share balance of the owner. However, if the owner has less withdrawable shares than its balance, the amount of burnt shares is lowered. As a result a migration could migrate an amount of assets inflated in comparison to the amount of burnt shares.

Vulnerability Detail

The function `migrateDeposit()` calculates the amount of assets to be migrated before lowering the amount of shares to be burnt:

```
    function migrateDeposit(
        address depositOwner
    ) external onlyOwner returns (uint256) {
        ...

        uint256 shares = balanceOf(depositOwner);
        uint256 assets = convertToAssets(shares);

        // We need to ensure the shares were not minted as debt, so they
        // are covered by the assets deposited to the vault.
        if (withdrawableShares[depositOwner] < shares) {
            shares = withdrawableShares[depositOwner];
        }

        if (shares == 0) {
            return 0;
        }

        // Adjust the withdrawable shares to exclude the shares that are being
        // migrated and keep the rest of shares associated with debt.
        withdrawableShares[depositOwner] = 0;

        // Burn the shares that are being migrated.
        _burn(depositOwner, shares);

        // Approve the assets to be transferred to the new contract.
        IERC20(asset()).forceApprove(migrateTo, assets);

        // Deposit the assets to the new contract.
        return IERC4626(migrateTo).deposit(assets, depositOwner);
    }
```

An attacker could mint 100 shares on account `A` and 1 share on account `B`, transfer the 100 shares from `A` to `B`. When `B` will be migrated, it will receive assets on `acreBTC` corresponding to 101 shares while only 1 share will be burnt. `B` can then send the 100 shares back to `A` that can withdraw assets corresponding to the 100 shares on `stBTC`.

Impact

Attackers can siphon all the funds from `stBTC` if they know when their migration will occur.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/stBTC.sol#L575-L613

Recommendation

Move the calculation `assets = convertToAssets(shares)` after having lowered the amount of shares to burn.

Response

The issue has been fixed as recommended by calculating the amount of assets after lowering the amount of shares to burn. The fix is present in commit `0xefdae0a`.

[High][Fixed] WithdrawalQueue requests could be stuck due to tBTCvault upgrade

Summary

When completing withdrawal requests via bridge, the withdrawal queue ensures that the owner of tBTC is the tbtcVault. However, the owner of tBTC can be updated when the protocol behind tBTC moves from one vault to the other. In that case, the pending withdrawal requests will never be processable.

Vulnerability Detail

The function `completeWithdrawalRequest()` checks that the owner of `tbtc` is the current value for `tbtcVault`:

```
    function completeWithdrawalRequest(
        uint256 _requestId,
        bytes calldata _tbtcRedemptionData
    ) external onlyMaintainer {
        // TBTC Token contract owner resolves to the TBTCVault contract.
        if (tbtc.owner() != tbtcVault) revert UnexpectedTbtcTokenOwner();
        ...
    }
```

An upgrade on `tBTCVault` will transfer the ownership to the new vault address (see `current tbtc vault code`):

```
contract TBTCVault ... {
    ...
    function finalizeUpgrade()
        external
        onlyOwner
        onlyAfterGovernanceDelay(upgradeInitiatedTimestamp)
    {
        emit UpgradeFinalized(newVault);
        // slither-disable-next-line reentrancy-no-eth
        tbtcToken.transferOwnership(newVault);
        ...
    }
    ...
}
```

There is no way to change the address for `tbtcVault` in the withdrawal queue resulting in a stuck contract.

Impact

If the tBTC vault is upgraded to a new address, the owner of tBTC will change and the withdrawal queue for bridge requests will no longer be processable until the code of the withdrawal queue is updated.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/midas/WithdrawalQueue.sol#L218

https://etherscan.io/address/0x9C070027cdC9dc8F82416B2e5314E11DFb4FE3CD#code

Recommendation

Add an admin function to the withdrawal queue to update the `tbtcVault` value.

Response

As of commit `0xefdae0a` this has been fixed by adding an only owner function `updateTbtcVault()` to update the `tbtcVault` value.

[Med][Fixed] Incorrect fee on acreBTC.redeem() and acreBTC.redeemAndBridge()

Summary

The function `redeem()` to redeem the input shares of `acreBTC` uses a different fee calculation than `previewRedeem()` when going through the withdrawal queue, resulting in a different amount of assets received by the user. This infringes ERC4626's recommendations.

This is also the case for `acreBTC.redeemAndBridge()` which relies on `WithdrawalQueue._finalizeRequestandTakeExitFee()` to take the exit fee.

Vulnerability Detail

The function `previewRedeem()` returns `assets - _feeOnTotal(assets, _exitFeeBasisPoints())` which uses `assets * feeBPS / (1 + feeBPS)` as a calculation for the fees:

```
    function previewRedeem(
        uint256 shares
    ) public view virtual override returns (uint256) {
        uint256 assets = super.previewRedeem(shares);
        return assets - _feeOnTotal(assets, _exitFeeBasisPoints());
    }

    function _feeOnTotal(
        uint256 assets,
        uint256 feeBasisPoints
    ) internal pure returns (uint256) {
        return
            assets.mulDiv(
                feeBasisPoints,
                feeBasisPoints + _BASIS_POINT_SCALE,
                Math.Rounding.Ceil
            );
    }
```

On the other hand, `redeem()` uses `_feeOnRaw()` to calculate the fees that will later be taken by the `WithdrawalQueue` which uses `assets * feeBPS`:

```
    function redeem(
        uint256 shares,
        address receiver,
        address owner
    ) public override returns (uint256) {
        uint256 assets = convertToAssets(shares);
        uint256 currentAssetsBalance = IERC20(asset()).balanceOf(address(this))
            ;

        if (assets > currentAssetsBalance && withdrawalQueue != address(0)) {
            // Check allowance if caller is not the owner
```

```
            if (msg.sender != owner) {
                _spendAllowance(owner, msg.sender, shares);
            }

            uint256 fee = _feeOnRaw(assets, _exitFeeBasisPoints());  // @audit
                that's the wrong fee function

            // Transfer shares to withdrawal queue using internal _transfer
            _transfer(owner, withdrawalQueue, shares);

            // Process redemption through queue
            WithdrawalQueue(withdrawalQueue).requestRedeem(
                shares,
                receiver,
                fee  // @audit-info exit fees in asset applied by
                    WithdrawalQueue
            );

            return assets;
        }

        return super.redeem(shares, receiver, owner);
    }

    function _feeOnRaw(
        uint256 assets,
        uint256 feeBasisPoints
    ) internal pure returns (uint256) {
        return
            assets.mulDiv(
                feeBasisPoints,
                _BASIS_POINT_SCALE,
                Math.Rounding.Ceil
            );
    }
```

Likewise, the function `WithdrawalQueue._finalizeRequestandTakeExitFee()` uses the incorrect calculation (and rounding) to compute the fees applied on the withdraw request:

```
    function _finalizeRequestandTakeExitFee(
        uint256 _requestId
    ) internal returns (uint256 tbtcAmount, uint256 exitFee) {
        WithdrawalRequest storage request = withdrawalRequests[_requestId];
        ...
        tbtcAmount = request.tbtcAmount; // cache the tbtc amount
        exitFee =
            (tbtcAmount * acrebtc.exitFeeBasisPoints()) /
            BASIS_POINT_SCALE;
        if (exitFee > 0) {
            IERC20(address(tbtc)).transfer(acrebtc.treasury(), exitFee);
        }
    }
```

Impact

The exit fee charged to the redeeming users will be higher than what is anticipated via `previewRedeem()`.

The vault does not strictly follow ERC4626.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/lib/ERC4626Fees.sol#L54-L59

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L346

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/midas/WithdrawalQueue.sol#L261-L263

Recommendation

Use the correct fee calculation and rounding in `acreBTC.redeem()` and `WithdrawalQueue._finalizeRequestandTakeExitFee()`.

Response

This as fixed as of commit `0xefdae0a` by using `_feeOnTotal()` in `acreBTC.redeem()` and `acreBTC.requestRedeemAndBridge()`. The fee is taken immediately and sent to the treasury by `WithdrawalQueue.requestRedeem()` and applied on finalization in the case of `withdrawalQueue.requestRedeemAndBridge()`

[Low][Ack] Migration from stBTC to acreBTC may incur entry fee twice

Summary

When migrating to acreBTC, stBTC calls `acreBTC.deposit()` to deposit the assets of the owner. This deposit function collects an entry fee if the `entryFeeBasisPoints` is not zero. However, the user already paid the entry fee collected by `stBTC` which seems like a redundant fee collection.

Vulnerability Detail

The function `stBTC.migrateDeposit()` migrates the assets of the deposit owner to the new `acreBTC` vault using the `deposit()` function:

```
    function migrateDeposit(
        address depositOwner
    ) external onlyOwner returns (uint256) {
        ...
        return IERC4626(migrateTo).deposit(assets, depositOwner);
    }
```

The `deposit()` function of `acreBTC` charges a fee on deposit since it inherits `ERC4626Fees`:

```
contract acreBTC is ERC4626Fees, PausableOwnable { ... }
```

Impact

If the value `acreBTC.entryFeeBasisPoints` is not zero, entry fees will be charge to the users during migration.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/stBTC.sol#L612

Recommendation

Document and acknowledge that the `entryFeeBasisPoints` will be zero during the migration.

Otherwise add a function `migrateDeposit()` to `acreBTC` that receives deposits without applying entry fees only callable during migration.

Response

The issue has been acknowledged as the `entryFeeBasisPoints` will be zero during migration. A comment has been added to `stBTC.migrateDeposit()`:

```
    /// @dev Before the migration, ensure that the entry fee in the new vault
    ///      is set to 0, to avoid depositor being charged the entry fee.
```

[Low][Ack] Limitation on shares during migration is broken in fungible stBTC

Summary

When migrating shares from `stBTC` to `acreBTC`, the amount of shares migrated is limited by `withdrawableShares[user]`. This value has no real meaning if `stBTC` is made fungible as it is no longer increased by deposits or decreased by withdrawals. This could prevent migration of certain shares for users.

Vulnerability Detail

The amount of shares that can be migrated from `stBTC` is limited by `withdrawableShares[user]`:

```
    function migrateDeposit(
        address depositOwner
    ) external onlyOwner returns (uint256) {
        ...

        uint256 shares = balanceOf(depositOwner);
        uint256 assets = convertToAssets(shares);

        // We need to ensure the shares were not minted as debt, so they
        // are covered by the assets deposited to the vault.
        if (withdrawableShares[depositOwner] < shares) {
            shares = withdrawableShares[depositOwner];
        }

        ...
    }
```

This value is no longer increased/decreased by deposit/withdrawals if the vault tokens are made fungible:

```
    function _deposit(
        address caller,
        address receiver,
        uint256 assets,
        uint256 shares
    ) internal virtual override {
        if (nonFungibleWithdrawalsEnabled) {
            withdrawableShares[receiver] += shares;
        }

        super._deposit(caller, receiver, assets, shares);
    }

    function _withdraw(
        address caller,
```

```
16          address receiver,
17          address owner,
18          uint256 assets,
19          uint256 shares
20      ) internal virtual override {
21          if (nonFungibleWithdrawalsEnabled) {
22              withdrawableShares[owner] -= shares;
23          }
24
25          super._withdraw(caller, receiver, owner, assets, shares);
26      }
```

Impact

The current value of `nonFungibleWithdrawalsEnabled` in the deployed `stBTC` is `True` (see etherscan). So long as it is not changed to `False` this is not an issue.

If fungible withdrawals are enabled, the migration of tokens may no longer be guaranteed.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/stBTC.sol#L575-L613

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/lib/ERC4626NonFungibleWithdrawals.sol#L87-L114

Recommendation

Use `currentDebt[depositOwner]` to reduce the amount of shares that can be migrated for the deposit owner.

Alternatively, acknowledge and document that the `stBTC` token will never be made fungible before being deactivated.

Response

The issue has been acknowledged as stBTC won't be made fungible before the migration. A comment has been added to `stBTC.migrateDeposit()`:

```
1       /// @dev The implementation assumes the stBTC token won't be made fungible
2       ///      before the migration and the migrated shares are limited to the
3       ///      withdrawable shares.
```

[Info] acreBTC is fungible

Summary

The contract `acreBTC` inherits from `ERC4626Fees` which inherits from `ERC4626NonFungibleWithdrawals`. However, it does not call `__ERC4626NonFungibleWithdrawals_init()` to disable the fungibility of the vault tokens.

Vulnerability Detail

The `initialize()` function of `acreBTC` does not call `__ERC4626NonFungibleWithdrawals_init()`:

```
    function initialize(IERC20 asset, address _treasury) public initializer {
        __ERC4626_init(asset);
        __ERC20_init("Acre Staked Bitcoin", "acreBTC"); // TODO: Confirm name
        __PausableOwnable_init(msg.sender, msg.sender);

        ...
    }
```

`__ERC4626NonFungibleWithdrawals_init()` is responsible for disabling fungibility of vault tokens:

```
    function __ERC4626NonFungibleWithdrawals_init() internal onlyInitializing {
        nonFungibleWithdrawalsEnabled = true;
    }
```

Impact

acreBTC vault tokens are fungible. This may be intended behaviour.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L106-L119

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/lib/ERC4626NonFungibleWithdrawals.sol#L36-L38

Recommendation

Acknowledge the issue if it is intended that tokens are fungible. Otherwise add a call to `__ERC4626NonFungibleWithdrawals_init()` in `acreBTC.initialize()`.

Alternatively, create a separate `ERC4626Fees` contract that does not inherit from `ERC4626NonFungibleWithdra` to be used by `acreBTC` to avoid confusion.

[Info] Non-fungible vaults can be made fungible with a wrapper contract

Summary

The contract `ERC4626NonFungibleWithdrawals` intends to make vault tokens non-fungible by preventing withdrawals from the vault by addresses that did not deposit tokens. However, this limitation can easily be circumvented by a wrapper contract that anyone can build on top of the vault.

Vulnerability Detail

The wrapper contract would deposit and withdraw in the name of its users and mint a fungible token in return that would be 1:1 redeemable for the non-fungible vault token.

Impact

I am not sure why non-fungibility is desired and cannot tell if this has any impact.

Recommendation

Reconsider the reason behind non-fungibility and acknowledge this issue.

[Info] Updating the fees should give enough notice for users to react

Summary

The owner of `stBTC` and `acreBTC` can update the fee parameter used for the calculation of entry and exit fee at any point. If a user deposits right after the entry fee is updated, they may pay a fee they did not anticipate. If the exit fee is updated, users may be forced to pay out an exit fee they did not sign up for.

Vulnerability Detail

The admin functions `updateEntryFeeBasisPoints()` and `updateExitFeeBasisPoints()` allow the owner to update the fee parameters instantly:

```
    function updateEntryFeeBasisPoints(
        uint256 newEntryFeeBasisPoints
    ) external onlyOwner {
        if (newEntryFeeBasisPoints > _BASIS_POINT_SCALE) {
            revert ExceedsMaxFeeBasisPoints();
        }
        entryFeeBasisPoints = newEntryFeeBasisPoints;

        emit EntryFeeBasisPointsUpdated(newEntryFeeBasisPoints);
    }

    ...
    function updateExitFeeBasisPoints(
        uint256 newExitFeeBasisPoints
    ) external onlyOwner {
        if (newExitFeeBasisPoints > _BASIS_POINT_SCALE) {
            revert ExceedsMaxFeeBasisPoints();
        }
        exitFeeBasisPoints = newExitFeeBasisPoints;

        emit ExitFeeBasisPointsUpdated(newExitFeeBasisPoints);
    }
```

No notice period is given to the users of the vault and they will have to pay the new fee.

Impact

Users may pay out fee they did not sign up for.

Code Snippets

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L177-L199

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/stBTC.sol#L243-L265

Recommendation

The functions to update the fee parameter should allow applying the change only after a certain delay (e.g. a week).

A more involved approach would be to store for each deposit the exit fee that the user agreed to pay out at the time of deposit and only apply this fee to that user.

Otherwise, acknowledge that enough warning will be given off-chain to users if the fees are ever changed or disable the fee change feature altogether.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

`dispatcher` not set in `acreBTC.initialize()`

The initializer of `acreBTC` does not set the `dispatcher` address. This address is used in `totalAssets()` to compute the assets owned by the vault. If the function `updateDispatcher()` is not called, the function `totalAssets()` will not work as well as all vault functions relying on it (most of the vault functions). I see no reason not to set the `dispatcher` in the initializer.

```
    function initialize(IERC20 asset, address _treasury) public initializer {
        __ERC4626_init(asset);
        __ERC20_init("Acre Staked Bitcoin", "acreBTC"); // TODO: Confirm name
        __PausableOwnable_init(msg.sender, msg.sender);

        if (address(_treasury) == address(0)) {
            revert ZeroAddress();
        }
        treasury = _treasury;

        minimumDepositAmount = 0.001 * 1e18; // 0.001 tBTC
        entryFeeBasisPoints = 0;
        exitFeeBasisPoints = 25; // 25 bps == 0.25%
    }

    function totalAssets() public view override returns (uint256) {
        return
            IERC20(asset()).balanceOf(address(this)) + dispatcher.totalAssets()
                ;
    }
```

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L106-L119

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L397-L400

stBTC mintDebt() and repayDebt() do not charge fees

The two debt-related functions `stBTC.mintDebt()` and `stBTC.repayDebt()` do not charge entry / exit fees as they directly mint / burn shares. This may be intended behaviour and is removed in `acreBTC` anyway.

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/stBTC.sol#L323-L398

Unclear fee segmentation

The `documentation` states that a 0.1% fee on deposits and 0.25% fee on withdrawals will be charged. In the code, `acreBTC` (and `stBTC`) use 0 fee on deposit and 0.25% on withdrawal. On the other hand, `BitcoinDepositor` uses 0.1% fee on deposit. If users can interact directly with `acreBTC` (or `stBTC`) they may avoid the fee from `BitcoinDepositor`.

I do not believe this is the intended user experience so I do not think this is an issue but it feels weird.

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L117

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/BitcoinDepositor.sol#L204

Potentially high gas save on vault redeems

During redeems, `acreBTC` transfers the redeemed shares to the withdrawal queue that immediately burns them. The vault could directly burn the shares to avoid impacting the storage of the withdrawal queue balance and save gas.

This can be ignored if the vault wants to be compatible with different withdrawal queues that may not burn the share.

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L384

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/acreBTC.sol#L349

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/midas/WithdrawalQueue.sol#L156

https://github.com/acre-btc/acre/blob/27c5f5f972a65d45a4a5f6182058e4791770b118/solidity/contracts/midas/WithdrawalQueue.sol#L178